



WEEK 09

By: Mir Faraz Ali
Programming Fundamentals



Void Functions:

- A void function is a function that performs an action but does NOT return any value.
- It only executes statements inside it.

Example:

```
#include <iostream>
using namespace std;
```

```
void greet() {           // void function
    cout << "Welcome Students!" << endl;
}
```

```
int main() {
    greet();              // calling the function
    return 0;
}
```

Call by Reference Parameters

- **Definition:**
- Call by Reference is a method of passing arguments where the function receives the reference (address) of actual variables.

Changes inside the function affect the original variables.

Why use it?

- To modify original data
- To save memory (no copying)

Example:

```
#include <iostream>
using namespace std;
void swapNums(int &a, int &b) { // Reference parameters
    int temp = a;
    a = b;
    b = temp;
}
int main() {
    int x = 5, y = 10;
    swapNums(x, y);

    cout << "After swapping: x = " << x << ", y = " << y << endl;
}
```


Using Procedural Abstraction

- Definition:
- Procedural Abstraction means breaking a large program into smaller functions, each doing one clear task.
- This makes the program organized, reusable, and easier to understand.

Using Procedural Abstraction

- Example:

Ordering Food at a Restaurant

You tell the waiter:

- “I want a pizza.”

You don't see:

- How dough is prepared
- How the sauce is made
- How the oven works

The procedure is hidden, but the result is delivered.

You just call the function: `orderFood()`.

Functions Calling Other Functions

- Definition:
 - A function can call another function inside it.
- This is used to divide tasks and reuse code.

Example:

```
void B() {  
    cout << "Inside Function B" << endl;  
}
```

```
void A() {  
    cout << "Inside Function A" << endl;  
    B();    // calling B inside A  
}
```

Introduction to Arrays

- Definition:

- An array is a collection of multiple values of the same data type stored in continuous memory locations.

Each value can be accessed using an index (starting from 0).

Uses

- Store multiple values under one name
 - Easy to loop and process data

Example:

Declaring and Defining Arrays:

```
int marks[5];           // declaration
```

```
int age[3] = {18, 20, 22}; // declaration + initialization
```

Arrays in Memory

- Definition:

Elements of an array are stored side by side in memory:

Example:

```
int arr[3] = {10, 20, 30};
```

Memory Layout:

Index:	0	1	2
--------	---	---	---

Value:	10	20	30
--------	----	----	----

Example:

```
#include <iostream>
using namespace std;
```

```
int main() {
    int nums[5] = {2, 4, 6, 8, 10};

    for (int i = 0; i < 5; i++) {
        cout << nums[i] << endl;
    }
}
```




End Of Class

